
Minor Assignment 1 - MLR56

Neural Nomads

Aryan Kumar
Medha Agarwal

Khushi Khandelwal
Om Bhartiya

Md Arsal Asif
Rounak Mishra

Question 1

1. First error:

$$\alpha_i \in [0, C]$$

Incorrect code

```
if newAlpha[i] < C:  
    newAlpha[i] = C  
if newAlpha[i] < 0:  
    newAlpha[i] = 0
```

Correct code

```
if newAlpha[i] > C:  
    newAlpha[i] = C  
if newAlpha[i] < 0:  
    newAlpha[i] = 0
```

2. Second error:

Incorrect equation for the updated weights.

Incorrect code

```
w_SDCM = w_SDCM - (newAlpha[i] + alpha[i]) * y[i] * x  
b_SDCM = b_SDCM - (newAlpha[i] + alpha[i]) * y[i]
```

Correct code

```
w_SDCM = w_SDCM + (newAlpha[i] - alpha[i]) * y[i] * x  
b_SDCM = b_SDCM + (newAlpha[i] - alpha[i]) * y[i]  
return newAlpha
```

3. Third error:

Incorrect objective function for primal and dual SDCM equations

$$L(\omega, \alpha, \beta) = \frac{|\omega|^2}{2} + \sum_{i=1}^m \alpha_i [1 - y_i(\omega^T x_i + b)]_+ \quad (1)$$

$$\sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m y_i y_j \alpha_i \alpha_j x_i^T x_j \quad \text{s.t.} \quad \alpha_i \geq 0 \quad \text{and} \quad \sum_{i=1}^m \alpha_i y_i = 0 \quad \text{and} \quad \omega = \sum_{i=1}^m \alpha_i y_i x_i \quad (2)$$

Incorrect code

```
def getCSVMPrimalDualObjVals( alpha ):
    global w_SDCM, b_SDCM
    hingeLoss = np.maximum( 1 - np.multiply( (X.dot( w_SDCM ) + b_SDCM), y ), 0 )
    # Recall that b is supposed to be treated as the last coordinate of w
    objPrimal = 0.5 * w_SDCM.dot( w_SDCM ) + 0.5 * b_SDCM * b_SDCM + C * np.sum(hingeLoss)
    objDual = np.sum( alpha ) - 0.5 * np.square( np.linalg.norm( w_SDCM ) ) - 0.5 * b_SDCM * b_SDCM

    return np.array( [objPrimal, objDual] )
```

Correct code

```
def getCSVMPrimalDualObjVals( alpha ):
    global w_SDCM, b_SDCM
    hingeLoss = np.maximum( 1 - np.multiply( (X.dot( w_SDCM ) + b_SDCM), y ), 0 )
    objPrimal = 0.5 * w_SDCM.dot( w_SDCM ) + C * np.sum(hingeLoss)
    objDual = np.sum( alpha ) - 0.5 * w_SDCM.dot(w_SDCM)
    return np.array( [objPrimal, objDual] )
```

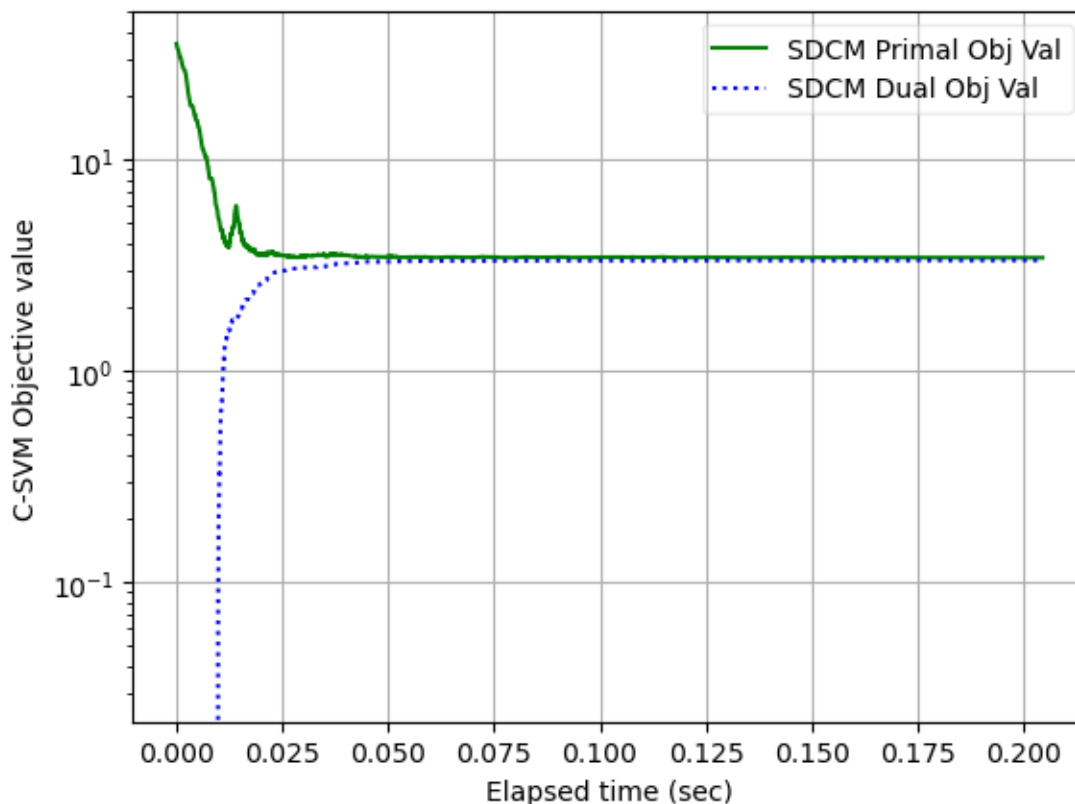
Question 2

2.1

Accuracy after code correction = 95.73030303030303%

2.2

Primal and Dual objective curves after code correction.



Question 3

3.1.1 Changing values of C

We change only the SVM regularisation constant C , from very small (10^{-5}) to very large (10^4), while keeping all other settings the same. This shows how C affects accuracy and the primal-dual objective curves.

We have already seen, in a C -SVM, the primal form is

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m [1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)]_+,$$

and the dual form is

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^m \alpha_i y_i = 0.$$

3.1.2 Behavior as C changes

- **Very small C :** Strong regularization. Margin is wide, many violations allowed. Few support vectors. Model underfits, so accuracy drops.
- **Moderate C :** Good balance. Enough penalty for violations to fit the data well, but still avoids overfitting. Usually gives the best accuracy.
- **Very large C :** Very strict on violations. Many support vectors hit the upper bound. Margin shrinks, and the model may overfit noisy data. Convergence can be slower or unstable.

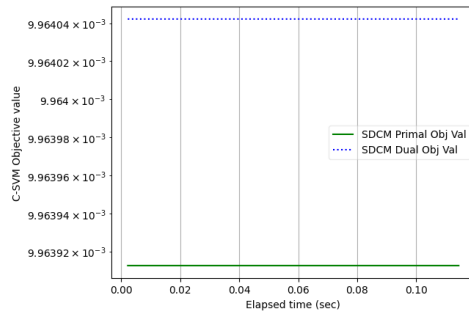
3.1.3 Results

We tested $C \in \{10^{-5}, 10^{-4}, \dots, 10^4\}$ on the corrected code. The results are summarized below:

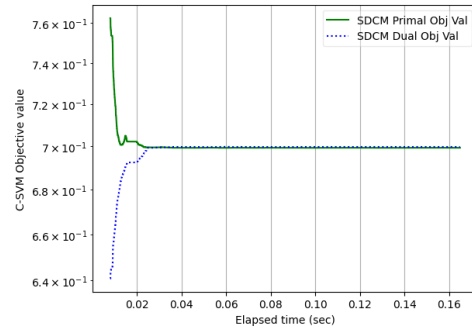
C	Testing Accuracy
10^{-5}	0.91666
10^{-3}	0.92417
10^{-1}	0.95734
1	0.94211
10	0.94868
10^2	0.90191
10^4	0.78017

Table 1: Effect of C on accuracy and objectives (all other hyperparameters fixed).

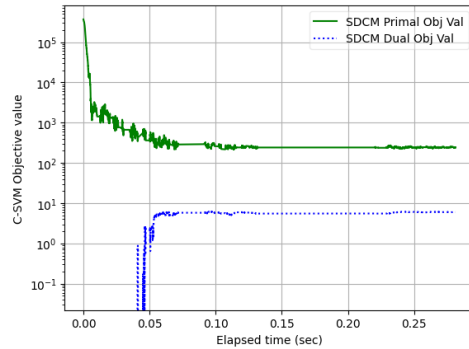
Representative plots of primal and dual curves are shown below for small, medium, and large C values, respectively.



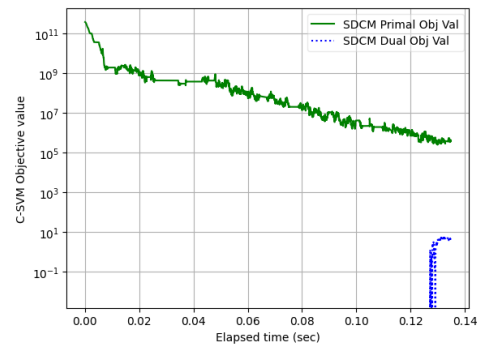
$c = 0.00001$



$c = 0.001$



$c = 1$



$c = 1000$

Takeaways

- Small C : wide margin, underfits.
- Medium C : best trade-off, highest accuracy.
- Large C : strict, overfits noisy data, unstable curves.

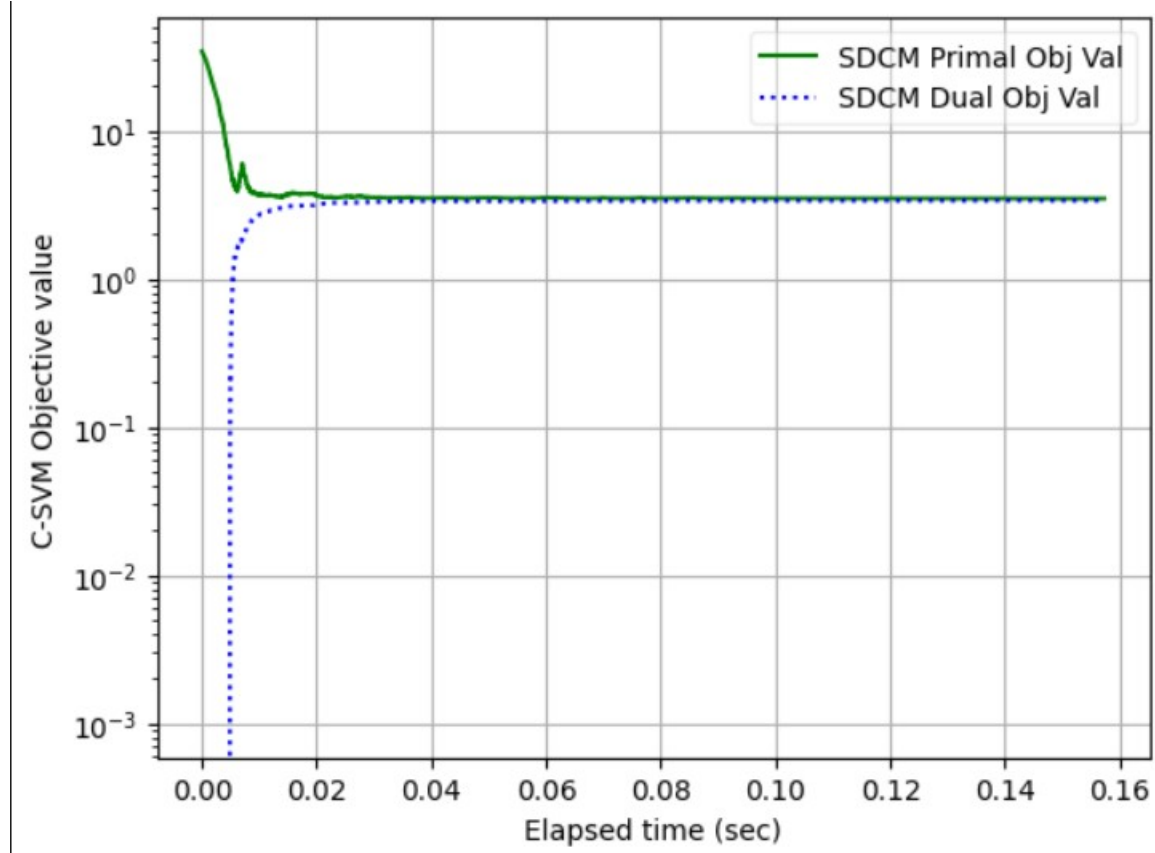
Question 4

4.1.1 The code for generating cyclic coordinates:

```
def getCyclicCoord( state ):
    idx = state[0]
    d = state[1]
    if idx >= d - 1: # if at the last coordinate
        idx = 0
    else:
        idx += 1
    state = (idx,d)
    curr = idx
    return (curr, state)
```

Accuracy = 95.06060606060606%

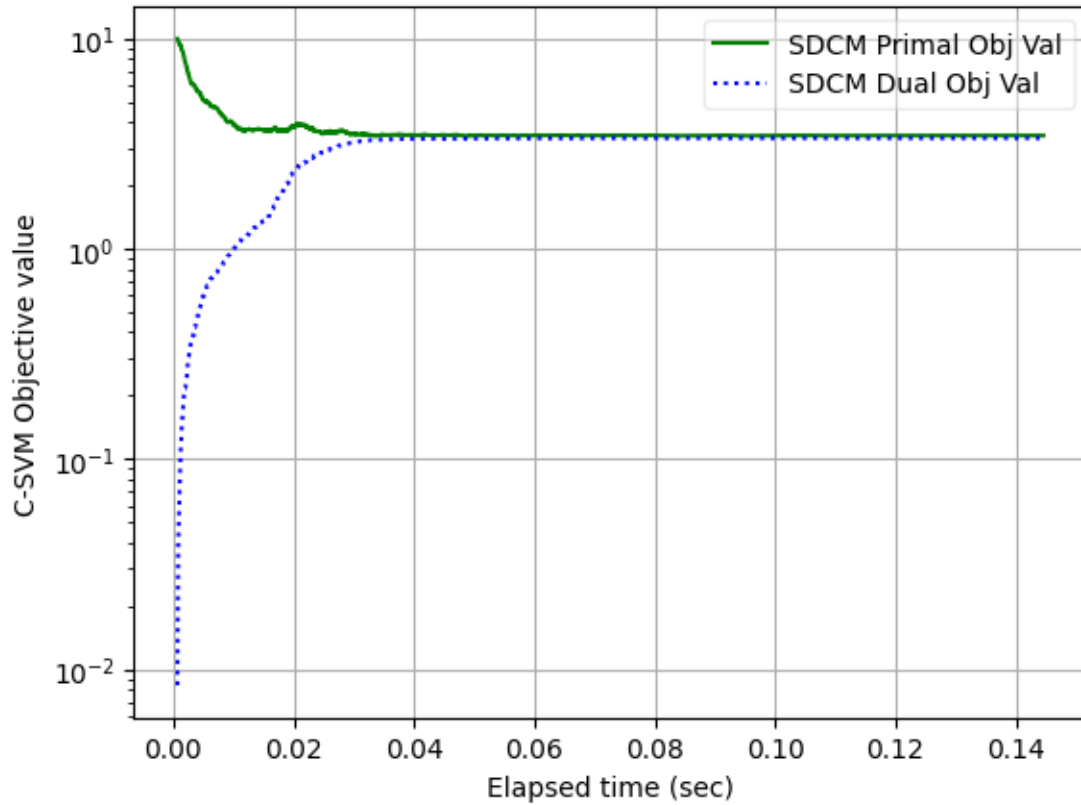
Primal and Dual objective curves



The primal and dual objective converges faster in terms of elapsed time. The accuracy decreased, because randomness avoids cyclic bias.

4.1.2 The code for generating random coordinates:

```
def getRandCoord( state ):
    d = state
    curr = np.random.randint(0, d) # random index from [0, d-1]
    return (curr, state)
```

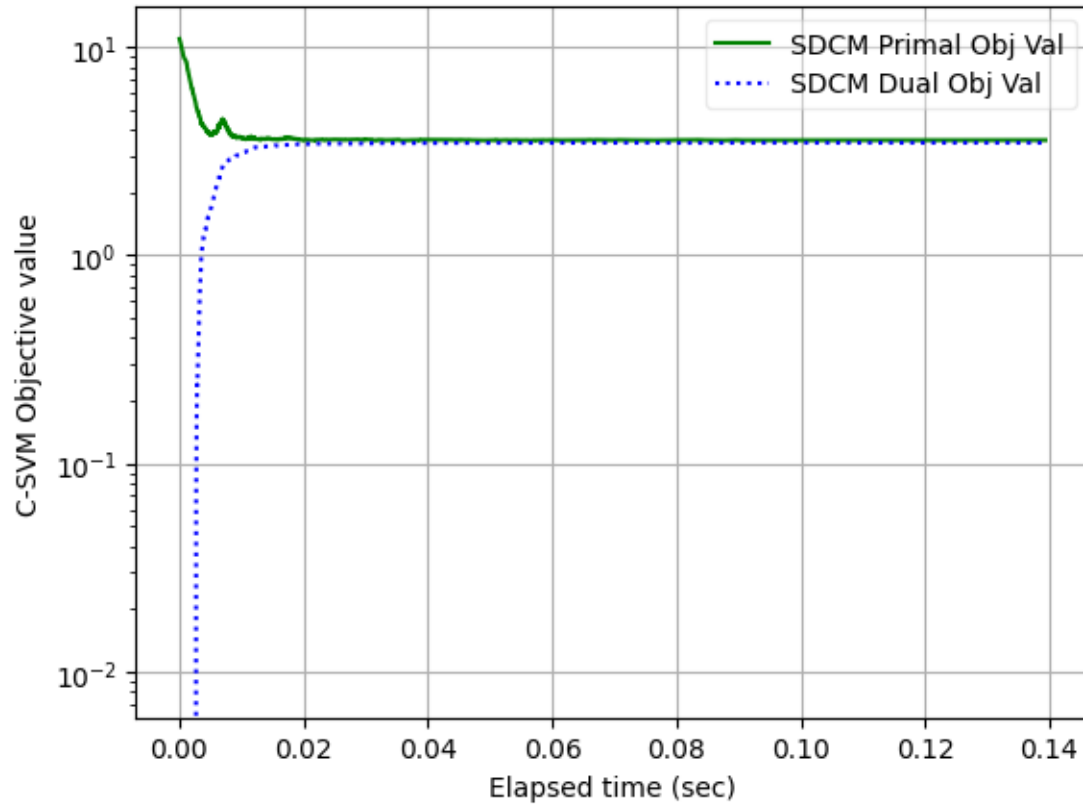



The primal and dual objective values start from a lower point (since all support for the margin is zero initially). The curves typically rise smoothly as the algorithm finds better values for alpha. Convergence is slower at the beginning compared to starting with all-C or random values, but the curves are usually smoother and more stable.

4.2.2 Changed hyperparameter initDual from all C to random initialization between 0 and C.

Accuracy = 94.41414141414142%

Primal and Dual objective curves

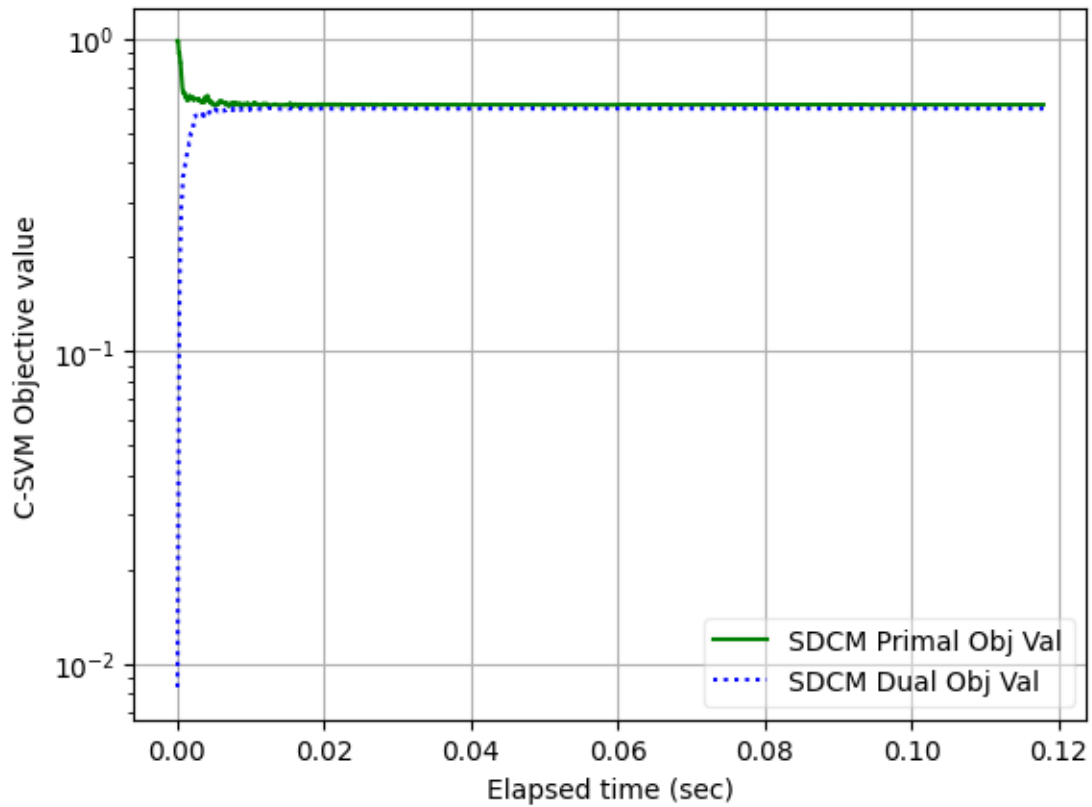


Random initialization of `initDual` changes the starting point of the optimization. This makes the convergence curves look more erratic at first, but the final accuracy and objective values are similar as the algorithm runs long enough.

4.4.1 Changed hyperparameter number of training points from 1000 to 100.

Accuracy = 83.24224224224224%

Primal and Dual objective curves

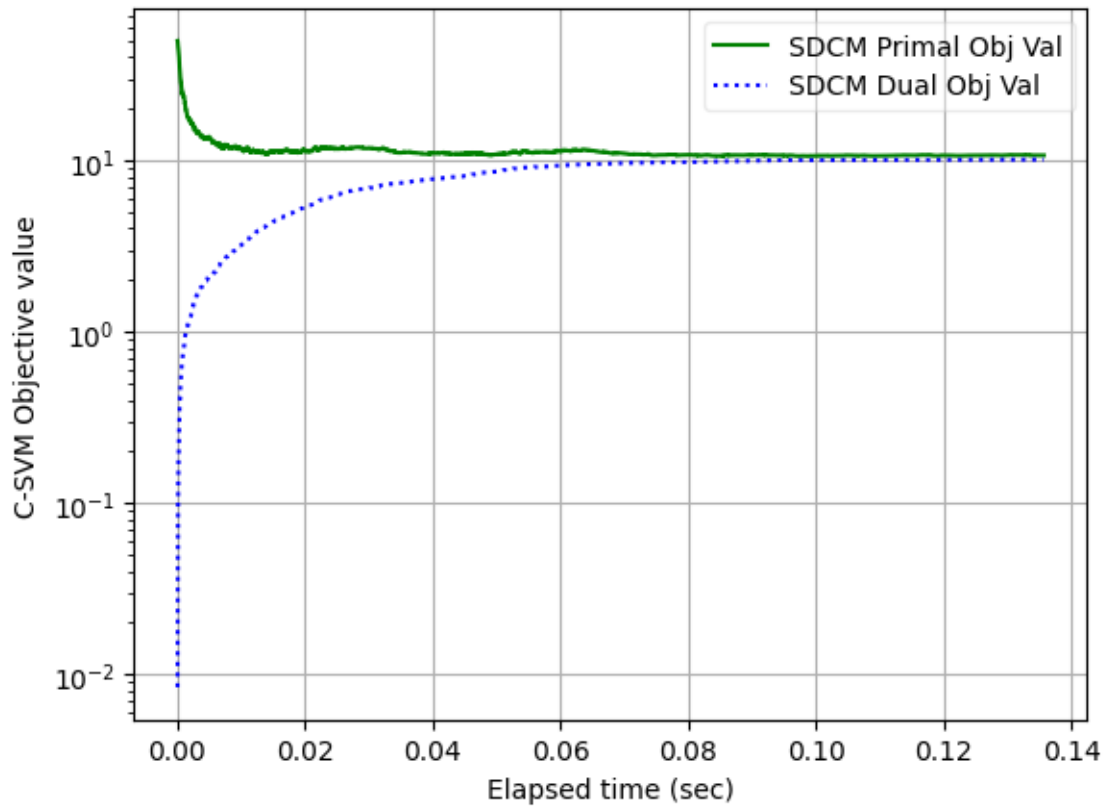


When the number of training points is decreased to 100, the SVM model has much less data to learn from, which results in lower accuracy . The convergence curves for the primal and dual objectives tend to stabilize quickly, but the final objective values are not be optimal, and the model will not generalize well to new data.

4.4.2 Changed hyperparameter number of training points from 1000 to 5000.

Accuracy = 98.22842105263158%

Primal and Dual objective curves



When the number of training points is increased to 5000, the model benefits from a richer dataset, leading to higher test accuracy and better generalization. The convergence curves take longer to stabilize because the optimization process involves more data, but the final objective values are more optimal.